Supplementary Notes: DeepSeek Architecture and Training

Hong Qin, Old Dominion University

Fall 2026

*Note. The material in these supplementary notes was covered in the 16-week version of this course and is **optional reading** for the 8-week offering. It assumes the Week 6 lecture notes on transformers (self-attention, positional encoding, the transformer block, and next-token training) and is provided for students who want to see how those ideas are extended in current research systems.*

Contents

1 Brief Introduction to the DeepSeek Model Family **2**

2 High-Level Architecture **2**

3 Review: Standard Transformer Self-Attention **3**
3.1 Mathematical Formulation . . . 3
3.2 Plain-Language Explanation . . . 4

4 Multi-Head Latent Attention (MLA) **4**
4.1 Latent K/V Definitions . . . 4
4.2 MLA with RoPE-Aware and NoPE Components (High-Level) . . . 5
4.3 Plain-Language Explanation of MLA . . . 5

5 Rotary Positional Embeddings (RoPE) **5**
5.1 Mathematical Definition . . . 6
5.2 Plain-Language Explanation of RoPE . . . 6

6 Mixture-of-Experts (DeepSeekMoE) and Router **7**
6.1 Mathematical Formulation . . . 7
6.2 Plain-Language Explanation of MoE and Router . . . 8

7 Reinforcement Learning and Policy Innovation in DeepSeek **8**
7.1 RL Training Pipeline for DeepSeek-R1 . . . 8
7.2 Group Relative Policy Optimization (GRPO): Objective and Advantages . . . 9
7.3 Reward Design and Emergent Reasoning . . . 11
7.4 Policy Innovation in a Nutshell . . . 11

8 Putting a DeepSeek Block Together **12**

9 Summary **13**

1 Brief Introduction to the DeepSeek Model Family

DeepSeek is a family of large language models (LLMs) designed to deliver competitive or superior performance to other frontier models while being significantly more efficient in training and inference.

Recent versions such as *DeepSeek-V3* and *DeepSeek-R1* are built on a decoder-only Transformer backbone but introduce three key architectural ideas:

1) **Multi-Head Latent Attention (MLA):** instead of storing a separate key and value tensor per attention head, DeepSeek compresses keys and values into a low-dimensional *latent* space and only caches these latent K/V tensors. At inference time, each head reconstructs its own keys and values from the shared latent representation. This drastically reduces KV-cache memory and bandwidth, enabling long contexts and large models with lower cost.

2) **DeepSeekMoE (Mixture-of-Experts):** the usual dense feed-forward network (FFN) is replaced by a sparse Mixture-of-Experts: many small expert FFNs plus a routing network that selects only a few experts per token. Variants such as fine-grained experts, shared experts, and improved load balancing give very high parameter capacity while keeping the per-token compute budget modest.

3) **Rotary Positional Embeddings (RoPE):** positional information is injected by applying position-dependent rotations to query and key vectors. In DeepSeek, RoPE is combined with MLA by splitting the representation into RoPE-aware and non-RoPE (NoPE) components, giving the model a flexible way to represent both positional and pure semantic information.

4) **Multi-Token Generation (MTG):** DeepSeek augments standard next-token prediction with mechanisms that allow the model to propose and verify several future tokens in a single forward pass. This includes parallel multi-token prediction heads during training and speculative or Medusa-style multi-token decoding during inference. MTG reduces autoregressive latency, increases throughput, and accelerates long-form reasoning and reinforcement-learning rollouts in DeepSeek-R1.

At a high level, a single DeepSeek Transformer block can be seen as:

$$\mathrm{Block}(X) = X + \mathrm{MLA}\big(\mathrm{LN}(X)\big) + \mathrm{MoE}\big(\mathrm{LN}(X + \mathrm{MLA}(\mathrm{LN}(X)))\big),$$

where LN denotes Layer Normalization, MLA is the attention module with latent K/V and RoPE, and MoE is the sparsely activated mixture-of-experts module.

2 High-Level Architecture

Let:

- L be the sequence length.
- d_{model} be the model dimension.
- H be the number of attention heads.
- $d_h = d_{\text{model}}/H$ be the per-head dimension.

Given a layer input

$$X \in \mathbb{R}^{L \times d_{\text{model}}},$$

a DeepSeek block proceeds in two main stages:

1) **Multi-Head Latent Attention (MLA)** with RoPE:

$$A = \text{MLA}(\text{LN}(X)) \in \mathbb{R}^{L \times d_{\text{model}}}.$$

2) **Mixture-of-Experts (MoE) FFN:**

$$M = \text{MoE}(\text{LN}(X + A)) \in \mathbb{R}^{L \times d_{\text{model}}}.$$

The output of the block is then:

$$Y = X + A + M.$$

Plain-language view:

- The attention part decides "what each token should look at" in the context, using MLA+RoPE efficiently.
- The MoE part applies several sparsely selected feed-forward "experts" to transform each token in a content-dependent way.

3 Review: Standard Transformer Self-Attention

For comparison, standard multi-head attention (MHA) proceeds as follows.

3.1 Mathematical Formulation

Input:

$$X \in \mathbb{R}^{L \times d_{\text{model}}}.$$

Linear projections:

$$\begin{aligned} Q &= XW_Q, \\ K &= XW_K, \\ V &= XW_V, \end{aligned}$$

where $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.

We split Q, K, V into H heads:

$$Q = [Q^{(1)}, \ldots, Q^{(H)}], \quad Q^{(h)} \in \mathbb{R}^{L \times d_h},$$

and similarly for $K^{(h)}, V^{(h)}$.

For head h:

$$\text{Attn}^{(h)}(X) = \text{softmax}\left(\frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_h}}\right) V^{(h)}.$$

We then concatenate and project:

$$\text{MHA}(X) = [\text{Attn}^{(1)}, \ldots, \text{Attn}^{(H)}]W_O,$$

with $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$.

3.2 Plain-Language Explanation

1) For each token, the model computes three versions of its representation: a *query*, a *key*, and a *value*.

2) For each head, it measures how similar the current token's query is to every other token's key (via dot product).

3) These similarities are turned into attention weights using softmax.

4) The head output is a weighted sum of all value vectors, using these attention weights.

5) All heads are concatenated and linearly mixed to form the final attention output.

The bottleneck: storing and streaming all keys and values for every head becomes very expensive for long sequences and large models.

4 Multi-Head Latent Attention (MLA)

MLA keeps the *queries* per head but compresses *keys* and *values* into a shared low-dimensional latent space, then reconstructs per-head K and V from that space. This greatly reduces KV-cache memory and bandwidth in autoregressive decoding and help the model process long sequences more efficiently.

4.1 Latent K/V Definitions

Let $d_\ell \ll d_{\text{model}}$ be the latent dimension. Starting from

$$X \in \mathbb{R}^{L \times d_{\text{model}}},$$

we compute:

Per-head queries. For each head h:

$$Q^{(h)} = XW_Q^{(h)}, \quad W_Q^{(h)} \in \mathbb{R}^{d_{\text{model}} \times d_h}.$$

Shared latent keys/values. We define head-independent latent K and V:

$$K_{\text{lat}} = XW_{K,\text{lat}} \in \mathbb{R}^{L \times d_\ell},$$
$$V_{\text{lat}} = XW_{V,\text{lat}} \in \mathbb{R}^{L \times d_\ell},$$

where $W_{K,\text{lat}}, W_{V,\text{lat}} \in \mathbb{R}^{d_{\text{model}} \times d_\ell}$.

Per-head reconstruction from latent space. For each head h, we have small "decoder" matrices:

$$U_K^{(h)}, U_V^{(h)} \in \mathbb{R}^{d_\ell \times d_h},$$

and reconstruct

$$K^{(h)} = K_{\text{lat}} U_K^{(h)} \in \mathbb{R}^{L \times d_h},$$
$$V^{(h)} = V_{\text{lat}} U_V^{(h)} \in \mathbb{R}^{L \times d_h}.$$

Attention computation. Attention itself is standard:

$$\mathrm{Attn}^{(h)}(X) = \mathrm{softmax}\left(\frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_h}}\right) V^{(h)}.$$

In autoregressive decoding, only K_{lat} and V_{lat} are cached. The full per-head $K^{(h)}, V^{(h)}$ are reconstructed on the fly via $U_K^{(h)}, U_V^{(h)}$.

4.2 MLA with RoPE-Aware and NoPE Components (High-Level)

DeepSeek further splits the representation into:

- A RoPE-aware part $(q_{\mathrm{rope}}^{(h)}, k_{\mathrm{rope}})$, which receives rotary position embeddings.
- A NoPE (no positional embedding) part $(q_{\mathrm{nope}}^{(h)}, k_{\mathrm{nope}}^{(h)})$, which stays purely semantic.

The attention score for head h is then based on a combination of these components, typically written schematically as

$$\alpha_{ij}^{(h)} \propto \left(\langle \tilde{q}_{\mathrm{rope},i}^{(h)}, \tilde{k}_{\mathrm{rope},j} \rangle + \langle q_{\mathrm{nope},i}^{(h)}, k_{\mathrm{nope},j}^{(h)} \rangle\right),$$

where $\tilde{q}$ and $\tilde{k}$ denote RoPE-rotated vectors (see Section 5).

4.3 Plain-Language Explanation of MLA

1) Instead of building a big key and value tensor for each head, the model builds a single compact *latent* key tensor K_{lat} and a single compact *latent* value tensor V_{lat} that all heads share.
2) During generation, only K_{lat} and V_{lat} are stored in the KV-cache; this dramatically shrinks the memory footprint.
3) Each attention head has its own small decoder matrices that convert the latent representation into its head-specific keys and values.
4) The attention mechanism (dot product, softmax, weighted sum of values) stays unchanged; the only difference is that keys and values are factored through a low-rank latent space.
5) As a result, DeepSeek can handle much longer sequences and larger models while keeping inference efficient.

5 Rotary Positional Embeddings (RoPE)

RoPE injects positional information into queries and keys by applying position-dependent 2D rotations in feature space.

5.1 Mathematical Definition

Consider a per-head vector $x \in \mathbb{R}^{d_h}$ (either a query or a key). Group its coordinates into 2D pairs:

$$x = (x_0, x_1, x_2, x_3, \ldots, x_{d_h-2}, x_{d_h-1}),$$

with pairs

$$(x_{2j}, x_{2j+1}), \quad j = 0, 1, \ldots, d_h/2 - 1.$$

For token position i (0-indexed), define a frequency ω_j and an angle

$$\omega_j = 10000^{-2j/d_h}, \qquad \theta_{i,j} = i \cdot \omega_j.$$

The RoPE-transformed pair at position i is

$$\begin{bmatrix} \tilde{x}_{2j} \\ \tilde{x}_{2j+1} \end{bmatrix} = \begin{bmatrix} \cos\theta_{i,j} & -\sin\theta_{i,j} \\ \sin\theta_{i,j} & \cos\theta_{i,j} \end{bmatrix} \begin{bmatrix} x_{2j} \\ x_{2j+1} \end{bmatrix}.$$

We denote this operation by

$$\tilde{x} = \mathrm{RoPE}(x, i).$$

Queries and keys are then replaced by their RoPE-rotated versions:

$$\tilde{q}_i^{(h)} = \mathrm{RoPE}(q_i^{(h)}, i), \qquad \tilde{k}_i^{(h)} = \mathrm{RoPE}(k_i^{(h)}, i).$$

The attention scores use $\tilde{q}$ and $\tilde{k}$:

$$\mathrm{score}_{ij}^{(h)} = \frac{\tilde{q}_i^{(h)} \cdot \tilde{k}_j^{(h)}}{\sqrt{d_h}}.$$

A key property of RoPE is that $\tilde{q}_i^\top \tilde{k}_j$ depends on the *relative* position $(i - j)$, enabling good extrapolation to long contexts.

5.2 Plain-Language Explanation of RoPE

1) View each pair of adjacent dimensions of a query or key vector as a point in a 2D plane. So, traditional sinusoidal PE is additive; ROPE is rotational .

2) For a token at position i, rotate every such 2D point by a position-dependent angle; different pairs use different rotation rates (frequencies).

3) When two rotated vectors from positions i and j are compared via dot product, the result now encodes both content similarity *and* the relative position $(i - j)$.

4) This gives the model a smooth, periodic representation of position that generalizes to sequences longer than those seen in training.

5) In DeepSeek's MLA, only part of the representation receives RoPE (RoPE part), while another part remains unrotated (NoPE), letting the model disentangle positional information from purely semantic dimensions.

6 Mixture-of-Experts (DeepSeekMoE) and Router

In a standard Transformer, the post-attention feed-forward network (FFN) is a single dense MLP. DeepSeek replaces this with a *sparse* Mixture-of-Experts (MoE): many experts, but only a small subset are active per token.

6.1 Mathematical Formulation

Suppose the hidden state entering the MoE at token t is $u_t \in \mathbb{R}^{d_{\text{model}}}$. There are N experts, each being an FFN:

$$\text{FFN}_i : \mathbb{R}^{d_{\text{model}}} \to \mathbb{R}^{d_{\text{model}}}, \quad i = 1, \ldots, N.$$

Routing scores. The router computes a score for each expert:

$$s_{i,t} = w_i^\top u_t + b_i, \quad i = 1, \ldots, N,$$

where $w_i \in \mathbb{R}^{d_{\text{model}}}$ and $b_i \in \mathbb{R}$ are learnable parameters.

Softmax probabilities.

$$p_{i,t} = \frac{\exp(s_{i,t})}{\sum_{j=1}^{N} \exp(s_{j,t})}.$$

Top-K gating. Let $\mathcal{T}_t$ be the set of indices of the top-K experts for token t (based on $p_{i,t}$). Define the normalized gates:

$$g_{i,t} = \begin{cases} \dfrac{p_{i,t}}{\sum_{j \in \mathcal{T}_t} p_{j,t}}, & i \in \mathcal{T}_t, \\ 0, & \text{otherwise.} \end{cases}$$

Expert outputs and aggregation. Each selected expert produces an output:

$$y_{i,t} = \text{FFN}_i(u_t).$$

The MoE output is the gated sum:

$$y_t = \sum_{i=1}^{N} g_{i,t}\, y_{i,t} = \sum_{i \in \mathcal{T}_t} g_{i,t}\, \text{FFN}_i(u_t).$$

DeepSeekMoE refinements (conceptual). DeepSeek introduces:

- *Fine-grained experts:* experts may be internally segmented into smaller sub-experts to increase diversity.
- *Shared experts:* a small subset of experts are always active (generalists) to capture common patterns and stabilize training.
- *Improved load balancing:* routing and training procedures reduce expert collapse and balance tokens across experts without heavy auxiliary losses.

6.2 Plain-Language Explanation of MoE and Router

1) Think of many small specialist networks (experts), each good at processing certain kinds of tokens or patterns.

2) For each token, the router looks at its hidden state and scores how suitable each expert is.

3) Only a few top-scoring experts (e.g. $K = 2$ or $K = 4$) are chosen for that token; all others are skipped.

4) The token is passed through those selected experts, producing several candidate transformed vectors.

5) The router then takes a weighted average of these candidate vectors to form the final MoE output for that token.

6) Because each token uses only a small subset of experts, the model can have a very large total number of parameters (high capacity) without increasing the per-token compute cost too much.

7 Reinforcement Learning and Policy Innovation in DeepSeek

Beyond architecture, DeepSeek models (especially DeepSeekMath and DeepSeek-R1) are trained with a distinctive reinforcement learning (RL) framework designed to improve reasoning while keeping training efficient. The key policy innovation is *Group Relative Policy Optimization* (GRPO), a critic-free variant of PPO that uses *relative* rewards within groups of samples.

7.1 RL Training Pipeline for DeepSeek-R1

Conceptually, DeepSeek-R1 is derived from a strong base model (e.g. DeepSeek-V3-Base) using several RL-related stages:

1) **DeepSeek-R1-Zero (pure RL on the base model).**

   - Start from the base model without any supervised fine-tuning.
   - Use GRPO with rule-based rewards that check final answer correctness and output format (for math, coding, logic tasks).
   - The model is trained purely via RL to solve verifiable problems; sophisticated reasoning behaviors such as longer thinking, self-reflection, and re-checking emerge automatically.

2) **DeepSeek-R1 with cold-start data.**

   - Collect a small but high-quality dataset of long chain-of-thought (CoT) solutions with human-designed formatting.
   - Supervised fine-tune the base model on this CoT data to obtain an initial actor that is more readable and user-friendly.

3) **Reasoning-oriented RL.**

   - Apply large-scale GRPO-based RL on reasoning-intensive tasks (math, coding, science, logic).

- Rewards combine final-answer correctness with auxiliary terms such as language consistency, encouraging both accuracy and readable reasoning.

4) **Rejection-sampling SFT.**

- From the RL-trained checkpoint, generate multiple candidate trajectories per prompt.
- Keep only correct and readable trajectories (via rule-based or model-based judgments).
- Use these high-quality trajectories to build a larger supervised dataset and fine-tune again, improving general skills (writing, dialogue, etc.).

5) **RL for all scenarios.**

- Run a second RL stage that mixes reasoning tasks with more general tasks.
- Combine rule-based rewards (for verifiable problems) with neural preference models that score helpfulness and harmlessness.
- Rewards focus mainly on the final answer for helpfulness, while safety rewards consider the whole reasoning trace and summary.

Overall, DeepSeek-R1 cycles between RL and supervised fine-tuning, gradually refining both reasoning ability and user-facing quality.

7.2 Group Relative Policy Optimization (GRPO): Objective and Advantages

GRPO is a policy gradient method related to PPO but designed for large LLMs:

- It **removes the critic network**. Instead of learning a separate value function, GRPO estimates advantages from *relative* scores within a group of responses.
- It **keeps PPO-style clipping** and a KL-penalty to stabilize updates.
- It is **memory-efficient** because no large critic model is required.

GRPO Objective

Let $P(Q)$ be a distribution over questions q. For each question q, GRPO samples a *group* of G responses from an old policy $\pi_{\theta_{\text{old}}}$:

$$\{o_1, o_2, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q).$$

Each response o_i receives a scalar reward r_i (for example, $r_i = 1$ if the final answer is correct, 0 otherwise, plus optional format/language bonuses). From these rewards, GRPO constructs a *group-normalized advantage*:

$$\bar{r} = \frac{1}{G}\sum_{j=1}^{G} r_j,$$

$$s_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \bar{r})^2 + \epsilon},$$

$$A_i = \frac{r_i - \bar{r}}{s_r},$$

where ϵ is a small constant to avoid division by zero.

Define the probability ratio between the new policy π_θ and the old policy $\pi_{\theta_{\text{old}}}$:

$$\rho_i(\theta) = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}.$$

GRPO maximizes the clipped, group-averaged objective with a KL regularizer:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim P(Q)} \left[\frac{1}{G} \sum_{i=1}^{G} \left(\min\big(\rho_i(\theta) A_i,\ \text{clip}(\rho_i(\theta), 1-\varepsilon, 1+\varepsilon)\, A_i\big) - \beta\, \text{KL}\big(\pi_\theta(\cdot \mid q) \,\|\, \pi_{\text{ref}}(\cdot \mid q)\big)\right)\right],$$

where ε and β are hyperparameters and π_{ref} is a reference policy (e.g. the base model or an SFT checkpoint).

Intuitively:

- If a response performs better than the group average ($A_i > 0$), GRPO pushes up its probability.
- If it performs worse ($A_i < 0$), GRPO pushes its probability down.
- The clipping on $\rho_i(\theta)$ prevents overly large policy updates, in the spirit of PPO.
- The KL term keeps the policy close to a reference model, stabilizing training.

Plain-Language Explanation of GRPO

1) For each question, sample several candidate answers from the *old* model.
2) Score every answer with a scalar reward (e.g. correct vs. incorrect, plus formatting/language bonuses).
3) Compare answers only *within the same group*:
   - Answers better than the group average get positive advantage.
   - Answers worse than the group average get negative advantage.
4) Compute how much more or less likely the new model wants to make each answer relative to the old one (this is the probability ratio ρ_i).
5) Increase the probability of above-average answers and decrease the probability of below-average answers, but clip the update so it does not move too far in one step.
6) Add a penalty if the new model drifts too far from a reference model (via the KL term).

This design replaces the complex value-function critic with a simple, group-based baseline and works well when rewards are sparse but verifiable (e.g. final answer correctness).

7.3 Reward Design and Emergent Reasoning

DeepSeek's RL pipeline uses relatively simple yet carefully chosen reward signals:

- **Accuracy rewards** that check if the final answer matches the ground truth:
  - For math, require the answer in a specific format (e.g. in a box or after a tag) and compare exactly.
  - For coding, compile and run tests; reward depends on passing test cases.
- **Format rewards** that enforce structural patterns such as:

$$\langle \text{think} \rangle \text{ reasoning process } \langle /\text{think} \rangle \langle \text{answer} \rangle \text{ final answer } \langle /\text{answer} \rangle.$$

- **Language and style rewards** that encourage output in a target language with readable formatting (e.g. markdown, clear summaries).
- **Preference rewards** in later stages, where a learned reward model judges helpfulness and harmlessness on open-ended tasks.

A notable empirical outcome is complex reasoning behaviors emerge without being hard-coded:

- The model learns to spend more "thinking time" (longer reasoning traces) on harder problems.
- It develops self-reflection patterns, such as revisiting previous steps when it suspects an error.
- It explores alternative solution strategies, guided only by correctness and simple format constraints.

In this sense, DeepSeek's RL approach is a policy innovation: rather than prescribing a specific style of reasoning, the training only rewards outcomes and minimal structure, and lets the policy autonomously discover effective internal strategies. (Instructor's aside: outcome-focused training also seems to work in children's development.)

7.4 Policy Innovation in a Nutshell

From a higher-level viewpoint, DeepSeek's RL and policy design innovate along three axes:

1) **Critic-free, group-relative optimization.** GRPO replaces traditional value-function critics with group-based baselines, making large-scale RL on LLMs more memory-efficient and easier to implement.
2) **Outcome-focused reward for reasoning.** Rewards mainly depend on verifiable final answers and light formatting constraints, leaving the reasoning process flexible. This encourages the model to invent its own reasoning heuristics.
3) **Multi-stage RL + SFT pipeline.** Alternating between RL (to explore reasoning strategies) and SFT (to distill good behaviors into cleaner policies) yields models that are both strong reasoners and user-friendly assistants.

8 Putting a DeepSeek Block Together

For a single layer ℓ and token t at position i, the forward flow is:

1) **Input:** hidden state $h_t^{(\ell)}$.

2) **LayerNorm + MLA projections:**

$$\hat{h}_t = \mathrm{LN}(h_t^{(\ell)}).$$

From $\hat{h}_t$ compute:

- per-head queries $q_t^{(h)}$,
- contributions to latent keys/values $K_{\mathrm{lat}}, V_{\mathrm{lat}}$.

3) **RoPE application:**

- For the RoPE-aware subspace, apply $\mathrm{RoPE}(\cdot, i)$ to queries and keys.
- The NoPE subspace remains unrotated, preserving pure semantic content.

4) **Attention:**

- For each head, reconstruct $K^{(h)}, V^{(h)}$ from $K_{\mathrm{lat}}, V_{\mathrm{lat}}$.
- Compute attention scores and outputs per head.
- Concatenate head outputs and project to get a_t.

5) **First residual connection:**

$$h_t^{(\ell,\mathrm{attn})} = h_t^{(\ell)} + a_t.$$

6) **LayerNorm + MoE:**

$$\hat{u}_t = \mathrm{LN}(h_t^{(\ell,\mathrm{attn})}).$$

The router scores experts on $\hat{u}_t$, picks top-K, and aggregates their FFN outputs into m_t.

7) **Second residual connection:**

$$h_t^{(\ell+1)} = h_t^{(\ell,\mathrm{attn})} + m_t.$$

8) **Repeat for next layer.**

Over many layers, MLA+RoPE handle efficient context integration, while the MoE provides high-capacity, specialized non-linear transformations; the RL training pipeline reshapes the overall policy to prioritize correct, readable, and aligned reasoning.

9 Summary

References: DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024, arXiv:2405.04434; DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024, arXiv:2412.19437; DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025, arXiv:2501.12948; Z. Shao et al., "DeepSeekMath," 2024, arXiv: 2402.03300 (the GRPO paper); J. Su et al., "RoFormer," 2021, arXiv:2104.09864.

- **Multi-Head Latent Attention (MLA)** factorizes keys and values through a low-dimensional shared latent space. Only latent K/V are cached, and per-head K/V are reconstructed on demand, greatly reducing memory and bandwidth.
- **RoPE** injects positional information by rotating 2D feature pairs in queries and keys with position-dependent angles. This yields a relative-position-aware similarity and supports long-context extrapolation.
- **DeepSeekMoE** replaces dense FFNs with a sparse mixture-of-experts: a router chooses a few experts per token, enabling very high parameter capacity with limited per-token computation.
- **GRPO-based RL** trains DeepSeek's policies using group-relative, critic-free policy optimization and simple, verifiable rewards. This encourages the emergence of strong reasoning behavior with efficient compute.
- A **DeepSeek block** combines MLA+RoPE for attention and DeepSeekMoE for the FFN, wrapped in LayerNorm and residual connections, while the RL pipeline (GRPO + staged SFT) turns the base architecture into a reasoning-focused model like DeepSeek-R1.